

This example is a great introduction to three important Node.js concepts:

- 1. Built-in modules**
- 2. Destructuring imports**
- 3. HTTP/HTTPS requests using streams and events**

1. The https Module

```
const { request, get } = require('https');
```

What is happening here?

Node.js has many built-in modules. One of them is the https module, which allows your application to communicate with HTTPS servers.

Think of it like:

```
const https = require('https');
```

Now the entire module is stored in https.

The module contains many methods:

```
https.request()  
https.get()  
https.Agent  
https.Server  
...
```

2. Destructuring

Instead of writing:

```
const https = require('https');
```

```
https.request(...)  
https.get(...)
```

you can extract only the functions you need:

```
const { request, get } = require('https');
```

This is JavaScript object destructuring.

Equivalent to:

```
const https = require('https');
```

```
const request = https.request;
```

```
const get = https.get;
```

Now you can simply write:

```
request(...)
```

```
get(...)
```

3. Understanding request()

```
const req = request('https://www.google.com', (res) => {
```

This creates an HTTPS request.

Think of it as:

"Node, prepare a request to Google."

At this point, the request has **not yet been sent**.

The Response Object (res)

When Google responds, Node passes the response into the callback:

```
(res) => {
```

res stands for response.

It contains:

- status code

- headers
- body data

Example:

```
console.log(res.statusCode);
```

might print:

200

meaning "success".

4. Why data Events?

```
res.on('data', (chunk) => {  
  console.log(`Data chunk: ${chunk}`);  
});
```

Web pages can be very large.

Instead of waiting for the entire page to arrive, Node receives it piece by piece.

These pieces are called **chunks**.

Imagine downloading a 10 MB file:

Chunk 1

Chunk 2

Chunk 3

Chunk 4

...

Each time a chunk arrives, Node emits a:

'data'

event.

Your callback runs for every chunk.

Example:

```
res.on('data', chunk => {  
  console.log(chunk);  
});
```

5. What is on()?

You have probably seen:

```
res.on(...)
```

This is because many Node objects are **Event Emitters**.

They emit events.

Examples:

Event	Meaning
data	New data arrived
end	No more data
error	Something went wrong

You listen for events using:

```
object.on(eventName, callback);
```

Example:

```
res.on('data', callback);
```

means:

"When data arrives, run this callback."

6. The end Event

```
res.on('end', () => {  
  console.log('No more data');  
});
```

```
});
```

After all chunks have arrived:

Chunk 1

Chunk 2

Chunk 3

Node emits:

'end'

which means:

"The response is finished."

Output:

No more data

7. Why Do We Need req.end()?

In the first example:

```
const req = request(...);
```

```
req.end();
```

Important

request() only creates the request.

It does **not** automatically send it.

You must finish the request yourself:

```
req.end();
```

Think of:

```
request()
```

as:

```
"Prepare an envelope."
```

and

```
req.end()
```

as:

```
"Seal and mail the envelope."
```

Without:

```
req.end();
```

the request may never be sent.

8. Why Doesn't `get()` Need `end()`?

The second example:

```
get('https://www.google.com', (res) => {  
  ...  
});
```

works immediately.

That's because `get()` is simply a convenience wrapper around `request()`.

Internally, it does something similar to:

```
const req = request(...);
```

```
req.end();
```

automatically.

So:

`get()`

is basically:

`request() + end()`

in one step.

9. When Should You Use `request()`?

Use `request()` when you need more control.

Example:

```
const req = request(options);
```

```
req.write(data);
```

```
req.write(moreData);
```

```
req.end();
```

This is common when:

- Sending form data
- Uploading files
- Making POST requests
- Sending JSON

Example:

```
req.write(JSON.stringify(user));
```

```
req.end();
```

10. When Should You Use `get()`?

Use `get()` when:

- You only want to retrieve data
- You are making a GET request
- You don't need to send a request body

Example:

```
get('https://api.github.com', callback);
```

Much simpler.

11. What Actually Happens Behind the Scenes?

When you run:

```
get('https://www.google.com', callback);
```

Node:

1. Creates a TCP connection.
2. Establishes HTTPS encryption.
3. Sends an HTTP GET request.
4. Waits for Google's response.
5. Receives chunks of data.
6. Emits data events.
7. Emits an end event when finished.

Flow:

```
get()
```

↓

Connect to Google

↓

Send GET request

↓

Receive response

↓

data event

↓

data event

↓

data event

↓

end event

12. A Better Real-World Example

Usually, you collect all chunks before using them:

```
const { get } = require('https');
```

```
get('https://www.google.com', (res) => {
```

```
  let data = '';
```

```
  res.on('data', (chunk) => {
```

```
    data += chunk;
```

```
  });
```

```
  res.on('end', () => {
```

```
    console.log(data);
```

```
  });
```

```
});
```

Why?

Because one chunk might contain only part of the HTML.

For example:

Chunk 1:

<html>

Chunk 2:

<body>

Chunk 3:

</body>

</html>

You often want the complete response before processing it.

Summary

Feature	request()	get()
Creates request	✓	✓
Sends request automatically	✗	✓
Requires end()	✓	✗
Can send request body	✓	✗ (normally)
Good for POST/PUT/PATCH	✓	✗
Good for simple GET requests	⚠	✓

A useful mental model is:

get()

=

request() + end()

So if you're simply fetching data from a URL, get() is easier. If you need to build a custom request and possibly send data, use request().